# Reflection Report

## Student Information Management System (SIMS) — AI-Assisted Laravel Development Project

**Project:** Student Information Management System (SIMS)
**Course:** Laravel Web Development
**Student:** Rexon Timbal
**Date:** November 1, 2026

---

## What Worked

- Splitting planning/verification from execution (see Agent Design Document) meant AI-reported "done" was never taken at face value — every step has an independent verification note in PROJECT_LOG.md (Tinker queries, browser checks, route-list dumps, re-running the test suite).
- Scoping each prompt to one module with an explicit, numbered requirements list (evolving from Step 1's looser prompt to Step 3's fully-specified one) produced code that needed little rework.
- Reusing an established pattern (Student module's migration/controller/view structure) for Subject and Enrollment kept the codebase consistent without re-explaining conventions each time.
- The final test suite (36 tests, 97 assertions, re-run after every subsequent visual change) caught zero regressions across three later redesign passes (Steps 4, 13, 14), which is direct evidence the earlier modules were built on solid foundations.

## What Failed / AI Mistakes

- **Destructive testing on real data (Step 7):** while testing cascade-delete behavior on the Enrollment module, the builder agent deleted the only seeded Student record to test the cascade, then tried to recreate it from memory — guessing the email, course, and year level instead of restoring the exact original values.
- **Silent route-name collision (Step 9):** Route::apiResource('students', ...) was added with default route names, which are the same namespace as the existing web Route::resource('students', ...). This silently overrode route('students.index'), which would have broken the sidebar navigation without throwing any error.
- **An authorization control that existed but was never wired up (Step 10):** AdminMiddleware was built correctly in Step 3, but no route in the entire app actually used it until Step 10 — every CRUD action, including delete, was reachable by any logged-in user regardless of role.

## Human Corrections Applied

- The Step 7 Student record was manually restored to its correct original values (rexon.timbal@sims.com, course BSIT, year level 4th Year) via the Students → Edit form, rather than leaving the guessed data in place.

- The Step 9 route collision was fixed by adding an explicit ->names('api.students') to the API resource route, decided and directed by the reviewer session once the collision was found, rather than leaving Laravel's default naming in place.
- The missing authorization wiring from Step 10 was identified entirely by self-review of the existing routes file (not flagged by the builder agent), and fixed with a route-group split that required zero changes to existing Blade route(...) calls.

## Lessons Learned

- An AI agent will treat "testing a destructive operation" and "performing a destructive operation" as the same action unless explicitly told otherwise. After Step 7, every later test against real data (e.g. the Step 8 zero-data dashboard check) was wrapped in a database transaction and rolled back, instead of trusting the agent to use non-destructive test data on its own.
- Code existing is not the same as code being applied. A correctly-written middleware sitting unused in the codebase will pass a code read-through and still leave a real security gap — it only becomes verifiable once something actually depends on it and that dependency is tested behaviorally.
- The most valuable catches in this project (the route-name collision, the unused middleware) came from checking the *running system* against the requirements, not from reading the AI's generated code in isolation.

# Reflection Questions

## 1. How did you reduce unnecessary AI context?

Each prompt was scoped to a single module or concern (Student, Subject, Enrollment, Dashboard, API, auth styling, etc.) across 14 discrete steps, so the builder agent only needed the current module's files in its working context, not the entire application history. Later modules (Subject, Enrollment) were told to follow the pattern already established by the Student module rather than having that pattern re-explained from scratch, which avoided re-deriving the same CRUD/validation structure in context repeatedly.

## 2. How did you avoid prompt repetition?

PROJECT_LOG.md served as persistent external memory: the exact prompt and verified result for every step was recorded once, so later steps and this reflection could reference prior decisions without restating them. Conventions (the shared rules() validation helper, the _form.blade.php partial structure, the themed data-table/empty-state CSS classes) were established once in Step 4/5 and referenced by name in later prompts instead of being redescribed.

## 3. How did you verify generated outputs?

By design, verification never relied on the builder agent's own report. A separate chat session independently re-checked results against ground truth after every step: querying the database directly via Tinker (role assignments, record counts), reading configuration files directly (bootstrap/app.php for the middleware alias), running php artisan route:list and php artisan test itself rather than trusting the reported counts, and performing live browser checks including a raw unauthenticated fetch() DELETE request to prove the Step 10 authorization boundary was a real server-side control and not just a hidden UI button.

## 4. How did you balance token cost and quality?

Work was decomposed into 14 sequential, single-purpose steps instead of one large end-to-end prompt, which kept each individual exchange focused and inexpensive while still producing a fully-featured system over the course of the project. Early steps (Step 1, 2) used looser, shorter prompts where the task was low-risk and easily checked; higher-risk steps (Step 3's authorization setup, Step 10's security fix) used longer, fully-specified prompts with explicit verification instructions built in, spending more tokens only where correctness mattered most. Visual-only changes late in the project (Steps 13, 14) were paired with a cheap sanity check — re-running the existing 36-test suite — rather than a full manual re-verification pass, since the tests already covered the functional surface those changes were not supposed to touch.